

# 项目配置文件与源码

## launch.json

```
{
    // 使用 IntelliSense 了解相关属性。
    // 悬停以查看现有属性的描述。
    // 欲了解更多信息，请访问：https://go.microsoft.com/fwlink/?linkid=830387
    "version": "0.2.0", // launch.json 配置文件的架构版本，固定为 0.2.0，一般不用改
    "configurations": [ // 调试配置列表，可以配置多个不同的调试任务
        {
            "name": "gcc.exe - 生成和调试活动文件", // 调试配置名称，会显示在“运行和调试”的下拉列表中
            "type": "cppdbg", // 调试器类型，C/C++ 使用 cppdbg，需要安装 C/C++ 扩展
            "request": "launch", // 请求类型：launch=启动并调试新程序；attach=附加到已经运行的进程
            "program": "${fileDirname}\\${fileBasenameNoExtension}.exe", // 要调试的可执行文件：当前文件所在目录\当前文件名去掉扩展名.exe
            "args": [], // 传给程序的命令行参数，例如 ["arg1", "arg2"]
            "stopAtEntry": false, // 是否在程序入口 main 处暂停：true=启动后停在 main；false=直接运行到断点
            "cwd": "${workspaceFolder}", // 程序运行时的工作目录：
            // ${workspaceFolder}=当前工作区根目录
            "environment": [], // 额外环境变量，例如
            // [{"name": "VAR", "value": "value"}]
            "externalConsole": false, // 是否使用外部控制台：false=使用 VS Code 集成终端/调试控制台；true=弹出独立控制台窗口
            "MIMode": "gdb", // 调试器后端模式，MinGW 使用 gdb
            "miDebuggerPath": "C:\\mingw64\\bin\\gdb.exe", // gdb 调试器可执行文件路径，按你的 MinGW 安装位置修改
            "setupCommands": [ // 启动 gdb 后自动执行的命令
                {
                    "description": "为 gdb 启用整齐打印", // 命令描述
                    "text": "-enable-pretty-printing", // 实际执行的 gdb 命令，用于美化 STL 等容器的显示
                    "ignoreFailures": true // 如果该命令执行失败，是否忽略并继续调试
                }
            ],
            "preLaunchTask": "C/C++: gcc.exe build active file" // 调试前先执行的任务名，必须与 tasks.json 中的 label 完全一致
        }
    ]
}

#include "stdio.h"
int average(int a,int b, int c){
    return (a+b+c)/3;
}
int square(int a){
    return a*a;
}
```

```
int variance(int a,int b,int c){
    int ave=average(a,b,c);
    return (square(a-ave)+square(b-ave)+square(c-ave))/3;
}
int score(int a,int b,int c){
    return 3*average(a,b,c)-variance(a,b,c)/3;
}

void rank(int zh1,int zh2,int zh3){
    if (zh1 >= zh2 && zh2 >= zh3) {
        printf("小明 > 小强 > 小林");
    } else if (zh1 >= zh3 && zh3 >= zh2) {
        printf("小明 > 小林 > 小强");
    } else if (zh2 >= zh1 && zh1 >= zh3) {
        printf("小强 > 小明 > 小林");
    } else if (zh2 >= zh3 && zh3 >= zh1) {
        printf("小强 > 小林 > 小明");
    } else if (zh3 >= zh1 && zh1 >= zh2) {
        printf("小林 > 小明 > 小强");
    } else { // zh3 >= zh2 && zh2 >= zh1
        printf("小林 > 小强 > 小明");
    }
}

int main(){
    int x1, x2, x3;
    int y1, y2, y3;
    int z1, z2, z3;

    printf("请输入小明的三项成绩（顺序为A B C,以一个空格为间隔）：");
    scanf("%d %d %d", &x1, &x2, &x3);
    printf("请输入小强的三项成绩（顺序为A B C,以一个空格为间隔）：");
    scanf("%d %d %d", &y1, &y2, &y3);
    printf("请输入小林的三项成绩（顺序为A B C,以一个空格为间隔）：");
    scanf("%d %d %d", &z1, &z2, &z3);
    int zh1=score(x1,x2,x3);
    int zh2=score(y1,y2,y3);
    int zh3=score(z1,z2,z3);
    rank( zh1, zh2, zh3);
    return 0;
}
```